

Student Record Management System

Project Report

1. Introduction

The Student Record Management System is a console-based application written in C++ that allows an institution to manage student academic records. It demonstrates the practical use of core data structures and algorithms, namely singly linked lists, searching techniques, sorting techniques, and statistical analysis, organized into clean, reusable modules.

2. Objectives

- Store and manage student records (roll number, name, and marks in 5 subjects).
- Provide efficient searching using both linear and binary search.
- Provide multiple sorting options directly on the linked list.
- Generate meaningful statistics such as class average and top performers.
- Persist records to disk and reload them across sessions.
- Demonstrate modular C++ design using separate header/implementation files.

3. System Architecture

The project is divided into independent modules, each with its own header (.h) and implementation (.cpp) file, following good software engineering practice of separation of concerns:

Module	Responsibility
Student	Represents a single student record and computes total, average, and grade.
LinkedList	Custom singly linked list that stores Student nodes; supports insert, delete, search, and traversal.
Searching	Linear search by roll number/name on the list, and binary search on a sorted array.
Sorting	Bubble sort (by roll number or name) and selection sort (by average), operating directly on list nodes.
Statistics	Computes class average, top/lowest student, pass count, and subject-wise averages.
Utility	Input validation, formatted console output, and file save/load for persistence.
main	Menu-driven driver program that ties all modules together.

4. Data Structure Design

Student records are stored in a singly linked list rather than an array, so the number of students can grow or shrink dynamically without needing to resize or pre-allocate memory. Each node holds one Student object and a pointer to the next node. Sorting algorithms operate directly on the list by swapping the data held in two nodes, which avoids the overhead of converting the list into an array and back.

5. Algorithms Used

Searching:

- Linear Search — $O(n)$: used for search by roll number and by name directly on the linked list.
- Binary Search — $O(\log n)$: demonstrated on a sorted array of students by roll number.

Sorting:

- Bubble Sort — $O(n^2)$: sorts by roll number or by name, ascending or descending.
- Selection Sort — $O(n^2)$: sorts by average marks, descending by default.

6. Grading Scheme

Average Marks	Grade
≥ 90	A
≥ 75	B
≥ 60	C
≥ 40	D
< 40	F

7. File Persistence Format

Student records are saved to a plain text file (students.dat) using a simple delimited format so records can be inspected or edited manually if needed:

```
rollNo|name|mark1,mark2,mark3,mark4,mark5
```

8. Sample Menu

1. Add Student
2. Display All Students
3. Search Student
4. Sort Students
5. Delete Student
6. Show Statistics
7. Save Data to File
8. Load Data from File
9. Exit

9. Conclusion

This project demonstrates a practical, modular implementation of fundamental data structures and algorithms in C++. The clear separation between the Student model, the linked list container, and the searching/sorting/statistics utilities makes the codebase easy to extend — for example, by adding new

sorting criteria, additional subjects, or a graphical interface on top of the same core logic.